L-10

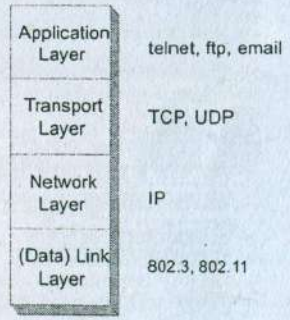
Ch-5.2

ECE 595
Computer Network Systems

TCP

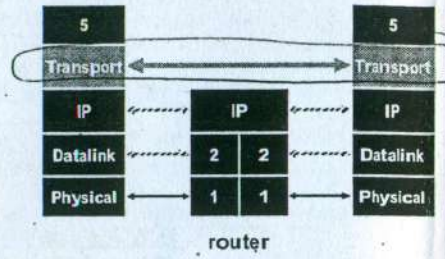
Reliable Transmission

Transport Layer



Transport Protocols Concern only End Hosts, not Routers

- Header generated by sender is interpreted only by the destination
- Routers view transport header as part of the payload



only at end nodes

Transport Layer in Internet

purpose of TL
① (De) mux to process
② value-added services

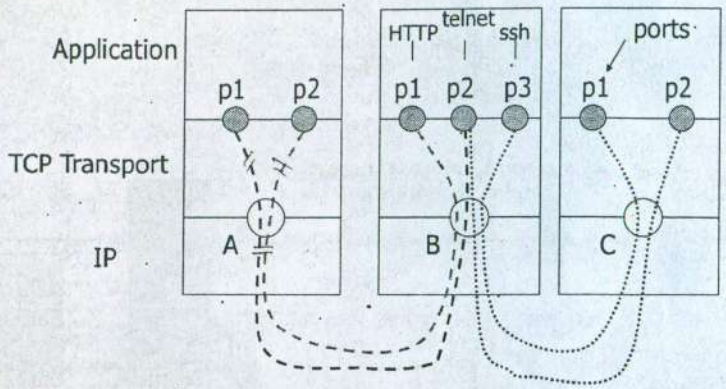
- Purpose 1: (De)multiplexing of data streams to different application processes
- Purpose 2: Provide value-added services that many applications want
  - Recall network layer in Internet provides a "Best-effort" service only, transport layer can add value to that
    - Application may want reliability, etc
  - No need to reinvent the wheel each time you write a new application

User Datagram Protocol (UDP)

- Connectionless datagram
  - Socket: SOCK_DGRAM
- Port number used for (de)multiplexing
  - port numbers = connection/application endpoint
- Adds end-to-end reliability through optional checksum
  - protects against data corruption errors between source and destination (links, switches/routers, bus)
  - does not protect against packet loss, duplication or reordering

0	16	32
Source Port	Dest. Port	
Length	Checksum	

Using Transport Layer Port Number to (De)multiplex traffic



In TCP, a data stream is identified by a set of numbers:
(Source Address, Destination Address, Source Port, Destination Port)

Using UDP

- Custom protocols/applications can be implemented on top of UDP
  - use the port addressing provided by UDP
  - implement own reliability, flow control, ordering, congestion control as it sees fit
- Examples:
  - remote procedure call
  - Multimedia streaming (real time protocol)
  - distributed computing communication libraries

Popular Transport Protocols

- UDP:
  - Barebones,minimal
  - Does not provide much functionality besides multiplexing
- TCP:
  - Elaborate, lots of additional functionality provided.

Transmission Control Protocol (TCP)

Error control + flow control + congestion control

- Reliable bidirectional in-order byte stream
  - Socket: SOCK_STREAM
- Lots of functionality
- Connection establishment.
  - Logical end-to-end connection, connection state to optimize performance
- Error control
  - Hide unreliability of the network layer from applications
  - Many types of errors: corruption, loss, duplication, reordering.
- End-to-end flow control and congestion control
  - Avoid flooding the receiver and network.

Errors: corruption → supported by UDP; loss, duplication, reordering — supported by TCP

Important TCP Flags (6 flag)

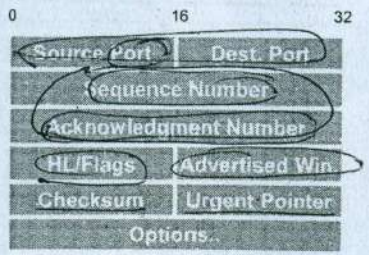
- SYN: Synchronize
  - Used when setting up connection
- FIN: Finish
  - Used when tearing down connection
- ACK
  - Acknowledging received data
- RESET:
  - Receiver wants to abort connection, as it received unexpected segment.
- Push and Urgent flags:
  - Not as commonly used.
  - Signify receiving process must be notified, or out-of-band data

Header

UDP → 8 bytes

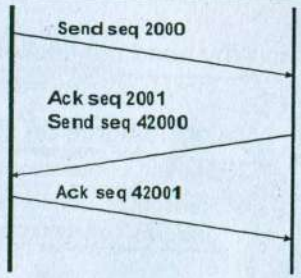
TCP → 20 bytes

TCP Header



- 20 bytes total
- Sequence Number, ACK, Advertised window: relate to TCP functionality for achieving reliable delivery.
- Sequence Number is for forward direction, ACK for reverse direction.
- HL: Specifies Header Length
- Flags: 6 flags in all
- Urgent pointer: not common TCP usage, used to signal certain data is "out-of-band" and must be processed immediately

Bidirectional Communication



- Each Side of Connection can Send *and* Receive
- What this Means
  - Maintain different sequence numbers for each direction
  - Single segment can contain new data for one direction, plus acknowledgement for other
    - But some contain only data & others only acknowledgement

Data পাঠাতে বা নেবার সময় sent seq# দেয় msg এ থাকবে
Data rcv করে বা নেবার সময় ACK# " " " " " am
(in header)

Ongoing Communication

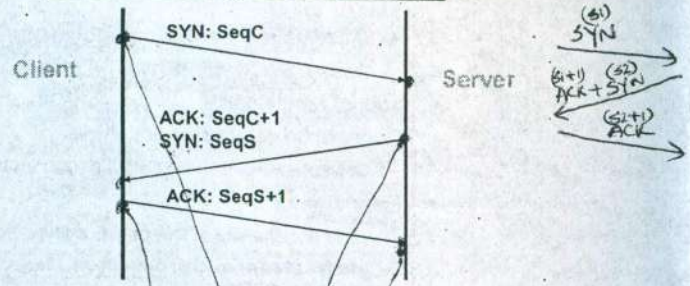
- Bidirectional Communication
  - Each side acts as sender & receiver
  - Every message contains acknowledgement of received sequence
    - Even if no new data have been received — ACK#
  - Every message advertises window size — Receiver Window
    - Size of its receiving window
  - Every message contains sent sequence number
    - Even if no new data being sent — Sent seq #
- When Does Sender Actually Send Message?
  - When sending buffer contains at least max. segment size (- header sizes) bytes — MSS → max^m segment size; IP payload এর সাথে
    MSS = MTU − (TCP and IP header size)
  - When application tells it
    - Set PUSH flag for last segment sent — max^m segment size এর আগে sending buffer এ জমা না হলেও app^n কখন সব send করবে, এজন্য push flag set করা হয়
  - When timer expires

3 সময় msg send
- max segment size এর সমান sending buffer এ জমা হলে।
- app^n push করলে।
- timer expire হলে।

Connection Setup

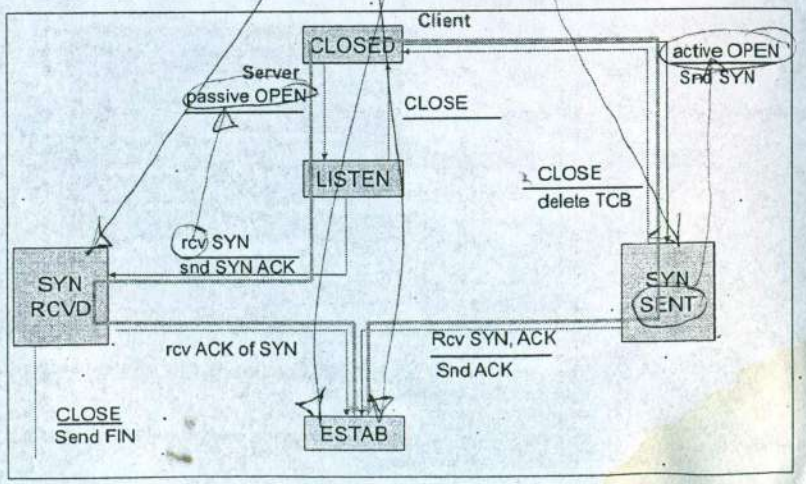
- Why need connection setup?
- Mainly to agree on starting sequence numbers
  - Starting sequence number is randomly chosen
  - Reason, to reduce the chance that sequence numbers of old and new connections from overlapping

Establishing Connection

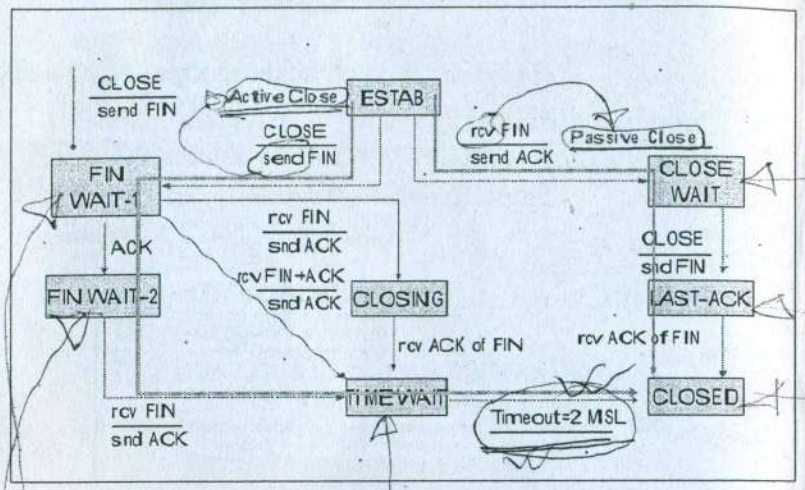


- Three-Way Handshake
  - Each side notifies other of starting sequence number it will use for sending
  - Each side acknowledges other's sequence number
    - SYN-ACK: Acknowledge sequence number + 1
  - Can combine second SYN with first ACK

TCP State Diagram: Connection Setup



State Diagram: Connection Tear-down



SYN-Flood Attacks

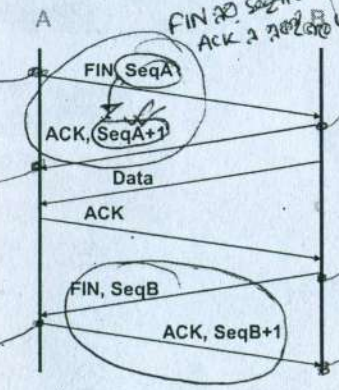
- Exploit 3-way handshake
- Attacker spoofs IP addresses, sends a large number of SYNs to the destination.
- Creates "partial unopened" state at the server
- Server sends SYN-ACK to (spoofed) addresses, to which no response is received.
- Rate at which SYNs sent much faster than rate at which server eliminates partial state.

Reliable Transmission

- Use two basic techniques:
  - Acknowledgements (ACKs)
  - Timeouts
- Two examples:
  - Stop-and-go
  - Sliding window

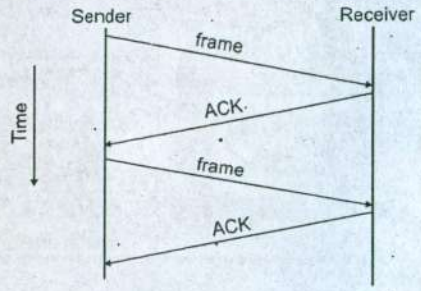
Tearing Down Connection

- Either Side Can Initiate Tear Down
  - Send FIN signal
  - "I'm not going to send any more data"
- Other Side Can Continue Sending Data
  - Half open connection
  - Must continue to acknowledge
- Acknowledging FIN
  - Acknowledge last sequence number + 1



Stop-and-Go

- Receiver: send an acknowledge (ACK) back to the sender upon receiving a packet (frame)
- Sender: excepting first packet, send a packet only upon receiving the ACK for the previous packet

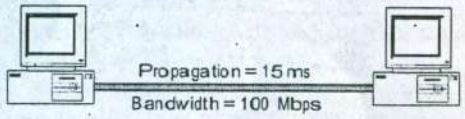


Stop-and-Go Disadvantage

Extremely inefficient (utilization of network capacity is very low)

- May lead to inefficient link utilization
- Example: assume
  - One-way propagation = 15 ms
  - Bandwidth = 100 Mbps
  - Packet size = 1000 bytes → transmit = (8*1000)/10^8 = 0.08ms
  - Neglect queue delay → Latency = approx. 15 ms; RTT = 30 ms

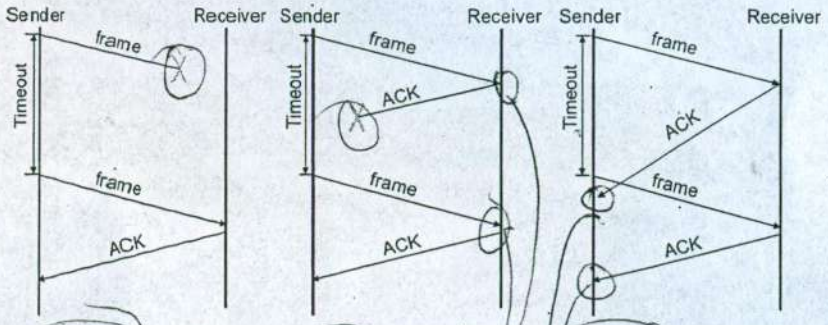
30 ms 중 0.08ms 만 (?)



$\frac{.08}{.08+30}$ = efficiency of useful work

$$\text{Utilization} = \frac{\text{achieved BW}}{\text{input BW}} = \frac{\frac{\text{size}}{\text{total time}}}{\frac{\text{size}}{\text{xmission time}}} = \frac{\text{xmission time}}{\text{total time}}$$

What Can Go Wrong?



⇒ From ACK 보고 duplicate 알 수 있음, seq#

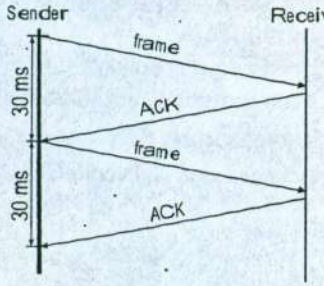
ACK#

Stop-and-Go Disadvantage (cont'd)

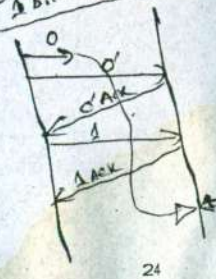
- Send a message every 30 ms → Throughput = 5.1 ms
  (8*1000)/0.03 = 0.2666 Mbps
- Thus, the protocol uses less than 0.3% of the link capacity!

30 ms 마다 1000*8 bits
5.1 ms 마다 1000*8 bits
$\frac{1000*8}{30\times10^{-3}}$ in ⇐ Throughput (1s 당 bit)

So, utilization = $\frac{\frac{1000*8}{30\times10^{-3}}}{10^8}$



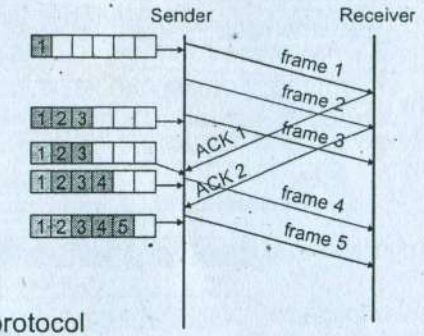
1 bit seq# 하면 되는 것?



⇒ delay 중에 마구 보내지 않고 1 bit seq# 하면 different 프레임 detect 할 수 있음

Sender buffer ⇒ for ack + App^n write
Receiver " ⇒ for out of order + App^n read
(effect of Network)

Sliding Window Approach



Sliding window protocol

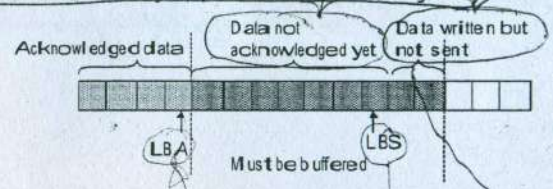
Send multiple packets without waiting for ACK.

Sliding window size: Max number of packets that can be sent without ACK being received (3 in figure)

Sliding Window Protocol: Sender

- TCP: operates at a byte stream level (rather than packet level)
- Sender maintains a window of sequence numbers (window of seq #)
  - SWS (sender window size) – maximum data that can be sent without receiving an ACK
  - LBA (last byte acknowledged)
  - LBS (last byte sent)
- TCP sender side socket buffer:
  - Data sent but not acknowledged
  - Data written by application but not sent yet

TCP sender buffer ⇒ 2 রকমের Data —
① send but not ack
② written by app^n but not sent (App^n fast হলে)

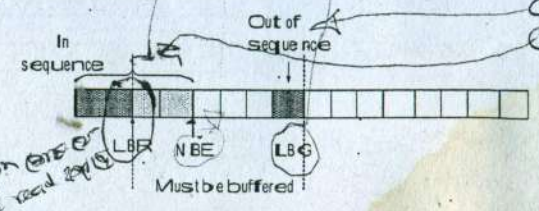


App^n ⇒ (3)

Sliding Window Protocol: Receiver

- TCP maintains a receive socket buffer
  - Stores data that arrived out-of-order (cannot be given to application yet)
  - Also stores data that arrived in-order but not yet read by application (slow process)
- Receiver maintains a window of sequence numbers
  - NBE (next byte expected – all previous bytes received in sequence)
  - LBR (last byte read by application)
  - LBG (last byte got by receiver)

TCP receiver buffer ⇒ 2 রকমের data —
① out of order
② in order but not read by app^n (App^n slow হলে)

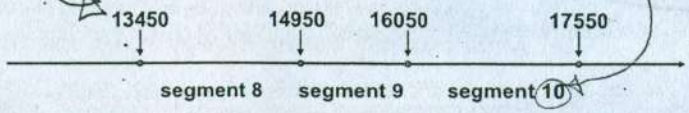


App^n এর byte read হয়নি

Sequence Numbers in TCP

- Each byte in byte stream is numbered.
  - 32 bit value
- TCP breaks up the byte stream in segments.
- Each segment has a sequence number.
  - Indicates where it fits in the byte stream

প্রতি segment 1 seq #

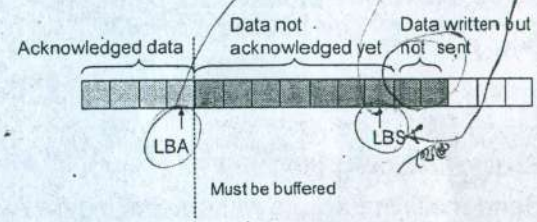


Effective Window = Advert win − (LBS − LBA)

Flow Control (sender side)

- Receiver throttles the sender by advertising a window no larger than the amount of data it can buffer
- Effective window at sender (limits how much it can send):
  Advertised window − (outstanding unacknowledged data) (LBS − LBA)
- Sending process can write to send-side TCP buffer only if space in sender side buffer – otherwise will block

effective window at sender = advertised window − outstanding data



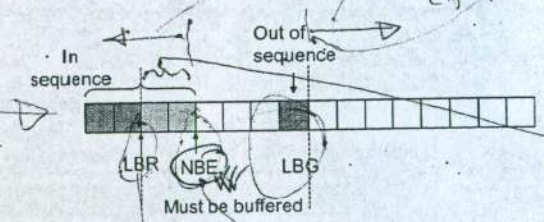
Seq# ⇒ 32 bits (concern: কত দ্রুত wrap around হবে while xmitting)
Adv windsize ⇒ 16 bits (concern: link কে full utilize করা হলে কতটুকু data থাকা উচিত ⇒ BW*delay).

Sliding Window Protocol: Receiver

- If incoming byte < NBE
  - Discard packet
- Else
  - Accept packet (provided it fits in the buffer)
  - ACK largest byte that all previous bytes were received

⇒ (NBE এর চেয়ে ছোট) এর (buffer এর বাইরে) হলে নিব না;
⇒ otherwise, নেব এবং max^m পর্যন্ত সবগুলি consecutive হলে, শেষ byte ACK

⇒ এ 2 এলাকার packet discard (if সেটা buffer এর end point এ)



Advertise window = MaxRcvBuffer − ((NBE − 1) − LBR)

[LBR বাড়লে বা NBE কমলে shrink হবে বোঝে। app^n যদি data read না করে incoming rate = App^n read rate হলে adver win = MaxRcv win হবে]

Buffer sizes

(BW * delay) to asure the pipe can be full

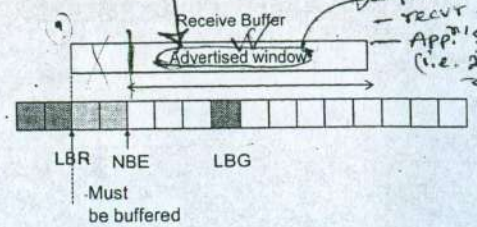
- Size of TCP socket buffers at sender and receiver side critical
- Would like this to be Bandwidth * Delay product of network, to ensure the pipe can be full.
- Must be tuned carefully for individual path.
- Traditionally, OSs used default socket buffer sizes that must be overriden by application programmer/end-user
  - "Tuning" in manual fashion tricky/hard.
- Research efforts at "auto-tuning" TCP buffers:
  - Automatic TCP Buffer Tuning
    Jeffrey Semke, Matthew Mathis, and Jamshid Mahdavi, SIGCOMM 1998
- More recently finding its way into some Operating Systems.

Flow Control

- Receiver throttles the sender by advertising a window no larger than the amount of data it can buffer
- Advertised Window depends on ⇒ Receiver যতটুকু buffer রাখতে পারে, max^m ততটুকু window advertise করে।
  - Size of receive buffer 16 bits
  - How fast receiver process can read the bytes from the buffer
- TCP sending window impacted by advertised window

→ Buffer empty হলে বড় buffer advertise
→ Buffer near full হলে small buffer advertise

⇒ Adv win = 0 হলে sender আর কোন byte পাঠাবে না to be stalled



depends on
- recvr buffer
- App^n's processing speed (i.e. recvr buffer এ কতটুকু already filled up)

Setting Timeouts

- Retransmission time-out should be set based on round-trip delay
- But round-trip delay different for each path!
- Must estimate RTT dynamically

Looping control — pckt id (Gnutella)

Flooding control — ~~pckt id~~ TTL (Gnutella); Seq # (OSPF)

Duplicacy check — Seq # (TCP)

Sequence Number Wraparound

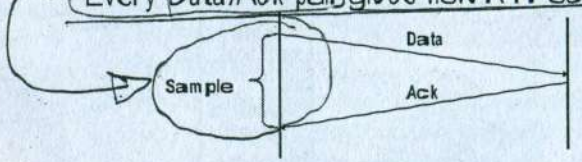
- TCP sequence numbers: 32 bits.
- Could wraparound
- Assuming packets remain in the network at most 120s, wrap around is a problem only for bandwidth above 286 Mbps
  - OK for most common networks
  - An issue for high-speed networks (e.g. special purpose scientific networks)
  - 120s: "Maximum segment lifetime": just a conservative estimate of how long a packet may linger in the network: not actual value.

V1: circular check
V2: 32 bits

Seq #
- OSPF ⇒ Flooding (flood link state for duplicacy check); A infinitely flood protect; Wrap around sol?
- TCP ⇒ duplicacy detect; Wrap around sol? ⇒ Not a problem with network of < 286 Mbps (Lifetime 120s)

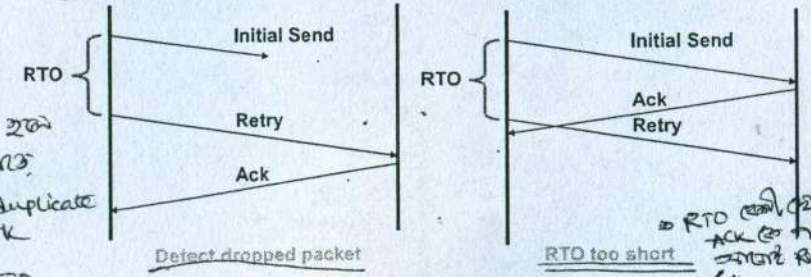
Round-trip Time Estimation

- Every Data/Ack pair gives new RTT estimate



- Can Get Lots of Short-Term Fluctuations

Setting Retransmission Timeout (RTO)

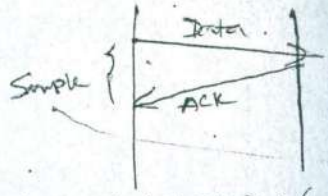


RTO too short ⇒ RTO expire হলে ACK না আসায় Rxmit হবে, So, many duplicate pkt send করে

RTO
- Too long: pckt drop হলে comm^n latency বাড়ে
- Too short: অহেতু duplicate pckt in network

General principle for RTO
RTO > 1 * RTT
[Otherwise, ACK আসার আগেই Rxmit হয়ে যাবে]

- Time between sending & resending segment
- Challenge
  - Too long: Add latency to communication when packets dropped
  - Too short: Send too many duplicate packets
  - General principle: Must be > 1 Round Trip Time (RTT)



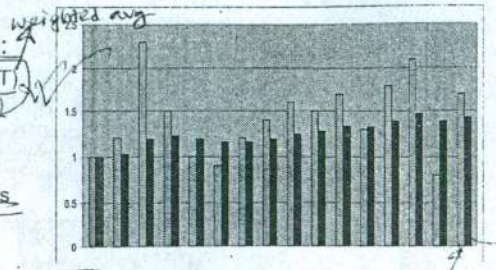
RTT → weighted avg → RTT = α (old RTT) + (1-α) new sample
α → .8 ~ .9 [.875 is used by most TCPs]
RTO → exponentially avg → RTO = b * RTT [where, b = 2]
↳ exponentially backed off for each expiry

Flow Control: Deals with ~~for~~ bottleneck at receiver
Congestion " : " " " " network

Original TCP Round-trip Estimator

RTO → exponentially avged

- Round trip times exponentially averaged:
  - New RTT = α (old RTT) + (1 - α) (new sample)
  - Recommended value for α: 0.8 - 0.9
    - 0.875 for most TCP's



- Retransmit timer set to (b * RTT), where b = 2
  - Every time timer expires, RTO exponentially backed-off
    → before Rexmit → Timeout (RTO) double
- Enhanced algorithm used in practice
  - [Jacobson/Karels: see textbook if interested]

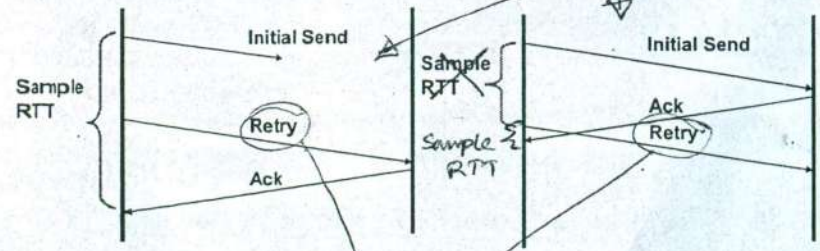
Karn/Partridge
① take RTT for only send without Rexmit
② Double RTO in case of each Rexmit.

① successful xmit, estimate RTT
RTO = b * RTT
② Rexmit, double RTO

Retry 후에 Sample RTO 안

Issue

Actually (pkt drop or Ack drop or other 2nd xmit)



What value of Sample RTT to use?

Solution: TCP simply neglects SampleRTT in such cases.
[Karn/Partridge algorithm]